

Riepilogo dei comandi – L02: shell e file system

Tutti i comandi della lezione, in un posto solo. Le opzioni elencate sono quelle che usiamo nel corso: `man < comando>` ha tutte le altre.

Vale per **Unix** – Linux e macOS, quindi la VM del corso. I comandi Windows corrispondenti sono nella tabella dei lucidi.

Questi comandi servono **a lavorare**, non all'esame: la prova è in Python, e quello che vi si può chiedere sta in `riepilogo_funzioni_python.md`. Qui trovate gli strumenti per guardare un file prima di aprirlo, che è un gesto che farete tutte le volte e che nessuna prova vi chiederà.

Orientarsi

Comando	Cosa fa	Opzioni utili
<code>pwd</code>	stampa la directory corrente	–
<code>ls</code>	elenca il contenuto di una directory	–l una riga per file con i dettagli · –a anche i file nascosti · –h dimensioni leggibili · –R anche le sottodirectory
<code>cd <dir></code>	cambia directory corrente	cd senza argomenti torna nella home · cd – torna dove eri prima

I simboli dei percorsi

Simbolo	Significato
/	la radice dell'albero, e il separatore fra le directory
~	la tua home directory
.	la directory corrente
..	la directory che la contiene
-	con cd: la directory precedente
*	qualunque sequenza di caratteri (<i>globbing</i>)

Un percorso **assoluto** inizia con / o ~ e vale da qualunque punto; uno **relativo** parte dalla directory corrente e cambia significato se ti sposti.

Chiedere aiuto, e scrivere meno

Comando o tasto	Cosa fa
man <comando>	apre il manuale (/parola cerca, q esce)
tab	completa il nome di comando, file o directory
frecce su e giù	scorrono i comandi già dati
ctrl-c	interrompe il comando in esecuzione
ctrl-d	chiude l'interprete interattivo (Python, less)

Creare, copiare, distruggere

Comando	Cosa fa	Opzioni utili
mkdir <dir>	crea una directory	-p crea anche le directory intermedie
touch <file>	crea un file vuoto, o aggiorna la data di modifica	—
cp <orig> <dest>	copia	-r per le directory
mv <orig> <dest>	sposta e rinomina	—
rm <file>	cancella un file, senza conferma e senza cestino	-r per le directory · -i chiede conferma
rmdir <dir>	cancella una directory solo se vuota	—

rmdir che protesta non è un ostacolo: è l'avviso che la directory contiene qualcosa che non hai guardato.

Guardare dentro un file

Comando	Cosa fa	Opzioni utili
cat <file>	stampa tutto il contenuto	—
less <file>	mostra il file una schermata per volta	/parola cerca · q esce
head <file>	le prime 10 righe	-n <N> le prime N
tail <file>	le ultime 10 righe	-n <N> le ultime N
wc <file>	conta righe, parole, caratteri	-l solo le righe

head -n 1 su un .csv mostra l'intestazione: è il modo più rapido di sapere quali colonne ha un file prima di aprirlo con pandas.

Produrre testo, e incanalarlo

Comando o simbolo	Cosa fa
echo "testo"	stampa il testo
seq <da> <a>	stampa una sequenza di numeri, uno per riga
comando > file	scrive l'output nel file, sovrascrivendolo
comando >> file	aggiunge l'output in fondo al file
comando1 \ comando2	manda l'output del primo nell'input del secondo (<i>pipe</i>)

Tagliare, ordinare, contare

I comandi che si mettono **dentro una pipe**: ciascuno fa una cosa sola, e si combinano.

Comando	Cosa fa	Opzioni utili
cut -d'<sep>' -f<n>	tiene solo la colonna n	-f1,6 più colonne · il separatore va fra apici
sort	ordina le righe, alfabeticamente	-u toglie i doppioli · -n ordina da numero · -r al contrario
sort -t'<sep>' -k<n>	ordina su una colonna precisa	si combina con -n e -r
uniq	schiaccia righe uguali e adiacenti	-c le conta · -d mostra solo i doppioli
tr '<a>' ''	sostituisce caratteri, uno a uno	tr ';' '\n' mette ogni campo su una riga
nl	numera le righe	utile dopo tr, per sapere il numero di una colonna
cat -A <file>	mostra i caratteri invisibili	\$ è il fine riga: rivela gli spazi in coda

uniq vuole l'ordinamento prima. Schiaccia solo righe uguali e *vicine*, quindi la tabella di frequenza è sempre
sort | uniq -c, mai uniq -c da solo.

Due esempi, sul file degli iscritti che ha il punto e virgola come separatore:

```
head -1 dati.csv | tr ';' '\n' | nl
```

numera le colonne, così sapete che numero dare a cut.

```
tail -n +2 dati.csv | cut -d';' -f1 | sort | uniq -c
```

conta quante righe ci sono per ciascun valore della prima colonna: è la tabella di frequenza, e `tail -n +2` serve a lasciare fuori l'intestazione, che altrimenti `cut` conta come un valore.

Cercare

Comando	Cosa cerca	Opzioni utili
<code>grep "testo" <file></code>	le righe che contengono il testo, dentro i file	-i ignora maiuscole · -c conta · -r scende nelle sottodirectory
<code>find <dir> -name '<modello>'</code>	i file il cui nome corrisponde al modello	il modello va fra apici: <code>'*.csv'</code>

Permessi ed esecuzione

Comando	Cosa fa
<code>ls -l</code>	mostra i permessi nella prima colonna (-rwxr-xr-x)
<code>chmod +x <file></code>	rende il file eseguibile
<code>chmod 644 <file></code>	lettura e scrittura al proprietario, sola lettura agli altri
<code>./<file></code>	esegue un file di questa directory

Archivi

Comando	Cosa fa	Opzioni utili
<code>zip <archivio.zip> <file></code>	comprime	-r per le directory
<code>unzip <archivio.zip></code>	decomprime nella directory corrente	-l elenca senza estrarre · -d <dir> estrae altrove · -q senza elenco

Python dalla shell

Comando	Cosa fa
<code>python <file>.py</code>	esegue lo script
<code>python</code>	apre l'interprete interattivo (<code>exit()</code> o <code>ctrl-d</code> per uscire)

Il percorso in `pd.read_csv("data/meteo.csv")` è relativo alla directory da cui hai lanciato Python, non a quella dove sta lo script. Il `FileNotFoundError` più comune del corso è un `pwd` sbagliato.

Le cinque trappole

1. **Spazi nei nomi** — `cd "Nuova Cartella"`, o meglio: niente spazi nei nomi.
2. **Maiuscole e minuscole contano** — `Dati.csv` $\neq$ `dati.csv` su Linux.
3. **`rm` è definitivo** — nessun cestino, nessuna conferma.
4. **`>` sovrascrive** — per aggiungere serve `>>`.
5. **I percorsi relativi dipendono da dove sei** — in caso di dubbio, `pwd`.

Gli strumenti moderni che sostituiscono alcuni di questi comandi (`rg` per `grep`, `fd` per `find`, ...) sono in `focus_ShellModerna.md`: materiale facoltativo, per chi finisce prima.